Danger

This is a “Hazardous Materials” module. You should **ONLY** use it if you’re 100% absolutely sure that you know what you’re doing because this module is full of land mines, dragons, and dinosaurs with laser guns.

ML-KEM key encapsulation

ML-KEM is a post-quantum key encapsulation mechanism based on module lattices, standardized in FIPS 203.

Encapsulation & Decapsulation

```
>>> from cryptography.hazmat.primitives.asymmetric.mlkem import MLKEM768PrivateKey
>>> private_key = MLKEM768PrivateKey.generate()
>>> public_key = private_key.public_key()
>>> shared_secret, ciphertext = public_key.encapsulate()
>>> recovered_secret = private_key.decapsulate(ciphertext)
>>> shared_secret == recovered_secret
True
```

Key interfaces

class **cryptography.hazmat.primitives.asymmetric.mlkem.MLKEM768PrivateKey** [source]

Added in version 47.0.0.

classmethod **generate**() [source]

Generate an ML-KEM-768 private key.

Returns: `MLKEM768PrivateKey`

Raises: cryptography.exceptions.UnsupportedAlgorithm – If ML-KEM-768 is not supported by the backend `cryptography` is using.

classmethod **from_seed_bytes**(*data*) [source]

Load an ML-KEM-768 private key from seed bytes.

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad

Raises:

- ValueError – If the seed is not 64 bytes.
- cryptography.exceptions.UnsupportedAlgorithm – If ML-KEM-768 is not supported by the backend `cryptography` is using.

```
>>> from cryptography.hazmat.primitives.asymmetric import mlkem
>>> private_key = mlkem.MLKEM768PrivateKey.generate()
>>> seed = private_key.private_bytes_raw()
>>> same_key = mlkem.MLKEM768PrivateKey.from_seed_bytes(seed)
```

`public_key`() [source]

Returns: `MLKEM768PublicKey`

`decapsulate`(*ciphertext*) [source]

Decapsulate a ciphertext using ML-KEM-768, returning the shared secret.

Parameters: **ciphertext** (bytes-like) – The ciphertext to decapsulate (1088 bytes).

Returns bytes: The shared secret (32 bytes).

Raises: ValueError – If the ciphertext is not the correct length.

`private_bytes`(*encoding*, *format*, *encryption_algorithm*) [source]

Allows serialization of the key to bytes. Encoding (`PEM` , `DER` , or `Raw`) and format (`PKCS8` or `Raw`) are chosen to define the exact serialization.

This method only returns the serialization of the seed form of the private key, never the expanded one.

Parameters:

- **encoding** – A value from the `Encoding` enum.
- **format** – A value from the `PrivateFormat` enum. If the `encoding` is `Raw` then `format` must be `Raw` , otherwise it must be `PKCS8` .
- **encryption_algorithm** – An instance of an object conforming to the `KeySerializationEncryption` interface.

Return bytes: Serialized key.

`private_bytes_raw`() [source]

Allows serialization of the key to raw bytes. This method is a convenience shortcut for calling `private_bytes()` with `Raw` encoding, `Raw` format, and `NoEncryption` .

This method only returns the seed form of the private key (64 bytes).

Return bytes: Raw key (64-byte seed).

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad

Added in version 47.0.0.

classmethod **from_public_bytes**(*data*) [source]

Parameters: **data** (*bytes*) – 1184 byte public key.

Returns: `MLKEM768PublicKey`

Raises:
- **ValueError** – If the public key is not 1184 bytes.
- **cryptography.exceptions.UnsupportedAlgorithm** – If ML-KEM-768 is not supported by the backend `cryptography` is using.

```
>>> from cryptography.hazmat.primitives import serialization
>>> from cryptography.hazmat.primitives.asymmetric import mlkem
>>> private_key = mlkem.MLKEM768PrivateKey.generate()
>>> public_key = private_key.public_key()
>>> public_bytes = public_key.public_bytes(
...     encoding=serialization.Encoding.Raw,
...     format=serialization.PublicFormat.Raw
... )
>>> loaded_public_key = mlkem.MLKEM768PublicKey.from_public_bytes(public_bytes)
```

encapsulate() [source]

Generate a shared secret and encapsulate it for this public key.

Returns: A `(shared_secret, ciphertext)` tuple where both values are `bytes`. The shared secret is 32 bytes and the ciphertext is 1088 bytes.

public_bytes(*encoding*, *format*) [source]

Allows serialization of the key to bytes. Encoding (`PEM` , `DER` , or `Raw`) and format (`SubjectPublicKeyInfo` or `Raw`) are chosen to define the exact serialization.

Parameters:
- **encoding** – A value from the `Encoding` enum.
- **format** – A value from the `PublicFormat` enum. If the `encoding` is `Raw` then `format` must be `Raw` , otherwise it must be `SubjectPublicKeyInfo` .

Returns bytes: The public key bytes.

public_bytes_raw() [source]

Allows serialization of the key to raw bytes. This method is a convenience shortcut for calling `public_bytes()` with `Raw` encoding and `Raw` format.

Return bytes: 1184-byte raw public key.

class **cryptography.hazmat.primitives.asymmetric.mlkem.MLKEM1024Priva**

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad

classmethod **generate**() [source]

Generate an ML-KEM-1024 private key.

Returns: `MLKEM1024PrivateKey`

Raises: cryptography.exceptions.UnsupportedAlgorithm – If ML-KEM-1024 is not supported by the backend `cryptography` is using.

classmethod **from_seed_bytes**(*data*) [source]

Load an ML-KEM-1024 private key from seed bytes.

Parameters: **data** (bytes-like) – 64 byte seed.

Returns: `MLKEM1024PrivateKey`

Raises:

- ValueError – If the seed is not 64 bytes.
- cryptography.exceptions.UnsupportedAlgorithm – If ML-KEM-1024 is not supported by the backend `cryptography` is using.

```
>>> from cryptography.hazmat.primitives.asymmetric import mlkem
>>> private_key = mlkem.MLKEM1024PrivateKey.generate()
>>> seed = private_key.private_bytes_raw()
>>> same_key = mlkem.MLKEM1024PrivateKey.from_seed_bytes(seed)
```

public_key() [source]

Returns: `MLKEM1024PublicKey`

decapsulate(*ciphertext*) [source]

Decapsulate a ciphertext using ML-KEM-1024, returning the shared secret.

Parameters: **ciphertext** (bytes-like) – The ciphertext to decapsulate (1568 bytes).

Returns bytes: The shared secret (32 bytes).

Raises: ValueError – If the ciphertext is not the correct length.

private_bytes(*encoding*, *format*, *encryption_algorithm*) [source]

Allows serialization of the key to bytes. Encoding (`PEM` , `DER` , or `Raw`) and format (`PKCS8` or `Raw`) are chosen to define the exact serialization.

This method only returns the serialization of the seed form of the private key, never the expanded one.

Parameters:

- **encoding** – A value from the `Encoding` enum.
- **format** – A value from the `PrivateFormat` enum

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad

- **encryption_algorithm** – An instance of an object conforming to the `KeySerializationEncryption` interface.

Return bytes: Serialized key.

`private_bytes_raw()` [source]

Allows serialization of the key to raw bytes. This method is a convenience shortcut for calling `private_bytes()` with `Raw` encoding, `Raw` format, and `NoEncryption`.

This method only returns the seed form of the private key (64 bytes).

Return bytes: Raw key (64-byte seed).

class `cryptography.hazmat.primitives.asymmetric.mlkem.MLKEM1024PublicKey` [source]

Added in version 47.0.0.

classmethod `from_public_bytes(data)` [source]

Parameters: **data** (*bytes*) – 1568 byte public key.

Returns: `MLKEM1024PublicKey`

Raises:

- **ValueError** – If the public key is not 1568 bytes.
- **cryptography.exceptions.UnsupportedAlgorithm** – If ML-KEM-1024 is not supported by the backend `cryptography` is using.

```
>>> from cryptography.hazmat.primitives import serialization
>>> from cryptography.hazmat.primitives.asymmetric import mlkem
>>> private_key = mlkem.MLKEM1024PrivateKey.generate()
>>> public_key = private_key.public_key()
>>> public_bytes = public_key.public_bytes(
...     encoding=serialization.Encoding.Raw,
...     format=serialization.PublicFormat.Raw
... )
>>> loaded_public_key = mlkem.MLKEM1024PublicKey.from_public_bytes(public_bytes)
```

`encapsulate()` [source]

Generate a shared secret and encapsulate it for this public key.

Returns: A `(shared_secret, ciphertext)` tuple where both values are `bytes`. The shared secret is 32 bytes and the ciphertext is 1568 bytes.

`public_bytes(encoding, format)` [source]

Allows serialization of the key to bytes. Encoding (`PEM` , `DER` , or `Raw`) and format (`SubjectPublicKeyInfo` or `Raw`) are chosen to define the exact serializa

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad

- **format** – A value from the `PublicFormat` enum. If the `encoding` is `Raw` then `format` must be `Raw` , otherwise it must be `SubjectPublicKeyInfo` .

Returns bytes: The public key bytes.

`public_bytes_raw()` [source]

Allows serialization of the key to raw bytes. This method is a convenience shortcut for calling `public_bytes()` with `Raw` encoding and `Raw` format.

Return bytes: 1568-byte raw public key.

latest

91% of mobile devs prefer mobile app security that covers the entire software development lifecycle.

Ads by EthicalAds

Close Ad